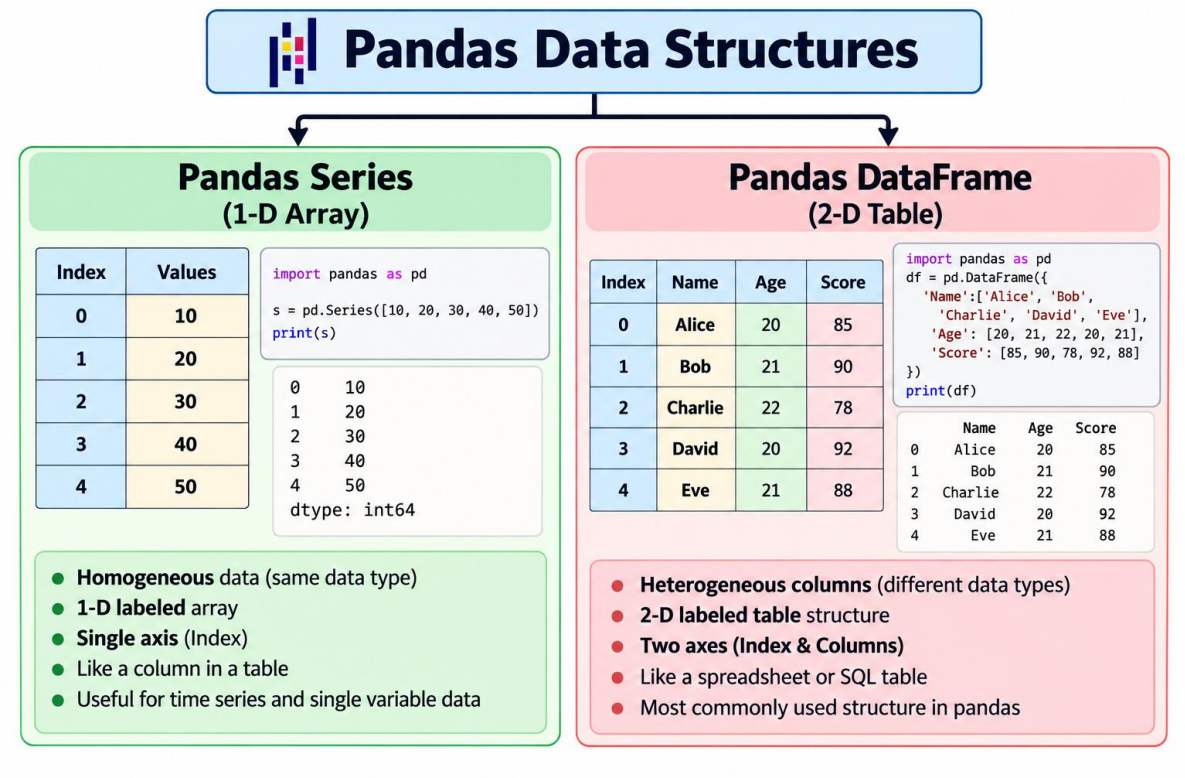
Introduction to Pandas Data Structures

Comprehensive Technical Guide & Reference

Pandas is the fundamental Python library for data manipulation, cleaning, and structured data analysis. Often described as providing "Excel-like capabilities inside Python," Pandas provides intuitive, high-performance data structures built directly on top of NumPy.

Understanding the core Data Structures in Pandas—Series, DataFrame, and Index—is essential before performing exploratory data analysis or building machine learning workflows.



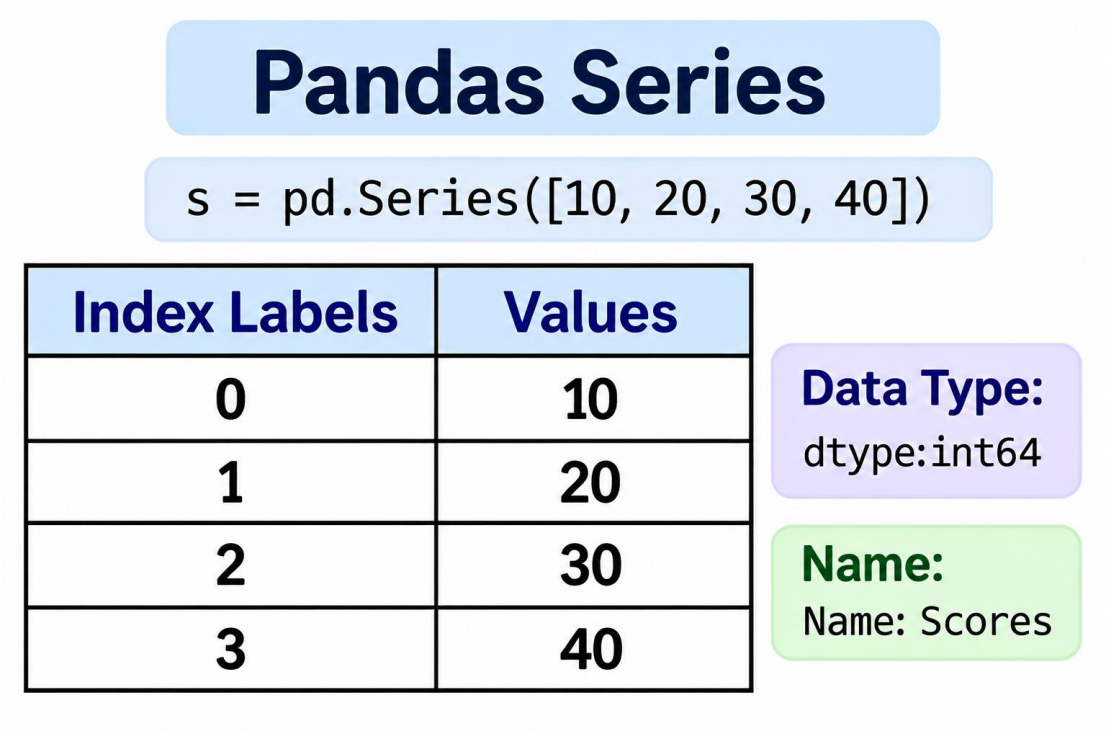
1. Pandas Series (1D Data Structure)

A **Series** is a one-dimensional labeled array capable of holding data of any type (integer, float, string, Python objects, etc.). It can be thought of as a single column in a spreadsheet or a specialized dictionary.

Key characteristics of a Series include:

- **Labeled Axis**: Every element has an associated index label.
- **Homogeneous Data**: All elements within a single Series typically share the same data type.
- **Automatic Alignment**: Operations between Series align data based on index labels rather than position.

Anatomy of a Series



Creating a Series

A Series can be constructed from Python lists, NumPy arrays, dictionaries, or scalar values.

```
import pandas as pd
import numpy as np

# 1. From a Python List (Default Integer Index)
s_list = pd.Series([10, 20, 30, 40], name="Scores")
print(s_list)

# 2. From a List with Custom String Index
s_custom = pd.Series([95, 88, 92], index=['Alice', 'Bob', 'Charlie'])
print(s_custom)

# 3. From a Python Dictionary (Keys become the Index)
data_dict = {'Math': 90, 'Science': 85, 'English': 88}
s_dict = pd.Series(data_dict)
print(s_dict)
```

Key Attributes of a Series

- `s.values`: Returns the underlying data as a NumPy ndarray.
- `s.index`: Returns the index (axis labels) object.
- `s.dtype`: Returns the data type of the underlying values.
- `s.shape`: Returns a tuple representing the dimensionality (e.g., `(4,)`).
- `s.hasnans`: Returns `True` if there are missing values (`NaN`).

Indexing and Accessing Series Data

Elements can be accessed using label-based indexing (`loc`), positional indexing (`iloc`), or boolean masking.

```
scores = pd.Series([95, 88, 92, 79], index=['Alice', 'Bob',
'Charlie', 'David'])

# Label Indexing
print(scores['Alice'])        # Returns 95
```

```
# Positional Indexing
print(scores.iloc[0])        # Returns 95

# Slicing
print(scores['Alice':'Charlie']) # Slices using labels (inclusive)

# Boolean Filtering
print(scores[scores > 90])   # Returns Alice (95) and Charlie (92)
```

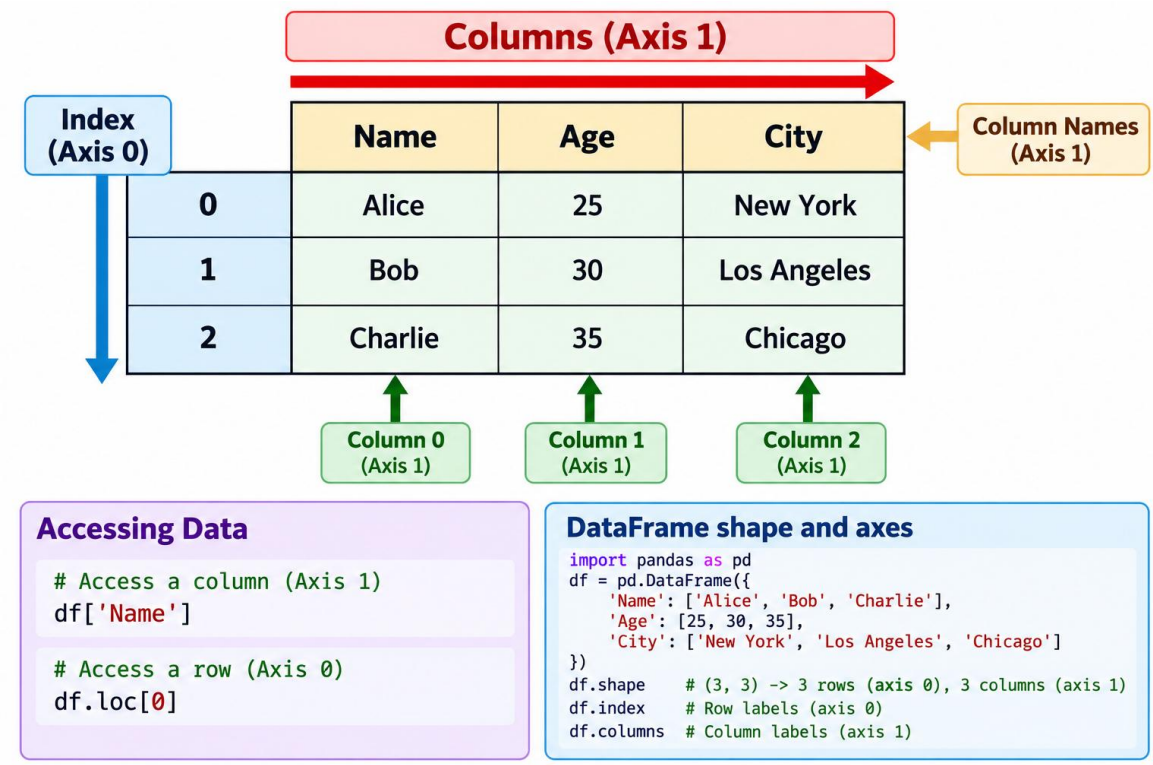
NOGLYPH 2. Pandas DataFrame (2D Data Structure)

A **DataFrame** is a two-dimensional, size-mutable, tabular data structure with labeled axes (rows and columns). It represents data in a spreadsheet, SQL database table, or a dictionary of Series objects.

Key characteristics of a DataFrame include:

- **Two-Dimensional Grid**: Organized into rows and columns.
- **Heterogeneous Column Types**: Different columns can store different data types (e.g., strings, integers, floats, booleans).
- **Dual Axis Labeling**: Rows are referenced by an **Index**, and columns are referenced by **Column Names**.

Anatomy of a DataFrame



Creating a DataFrame

DataFrames can be instantiated using various data inputs, including dictionaries, lists of lists, 2D NumPy arrays, or by reading external files (CSV, Excel, SQL).

```
import pandas as pd

# 1. From a Dictionary of Lists
data_dict = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['New York', 'Los Angeles', 'Chicago']
}
df_dict = pd.DataFrame(data_dict)
print(df_dict)
```

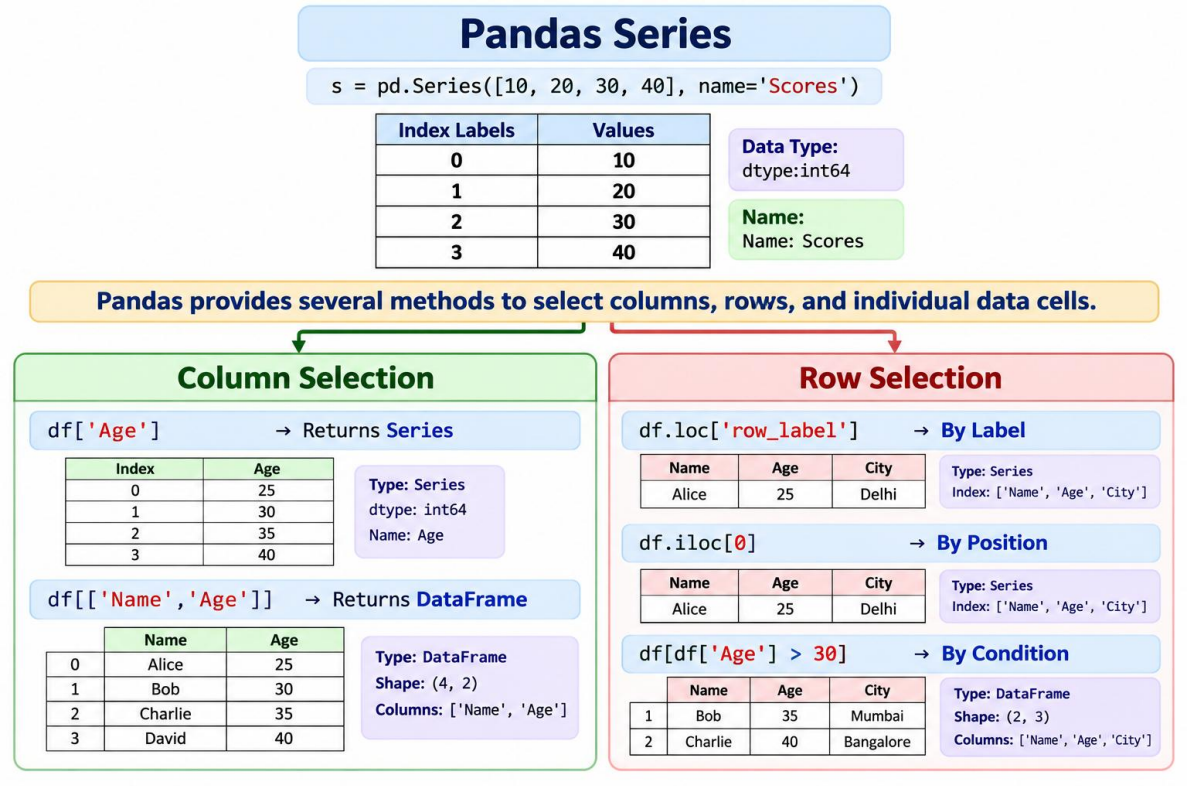
```
# 2. From a List of Lists (with explicit Column Headers)
data_list = [
    ['Alice', 25, 'New York'],
    ['Bob', 30, 'Los Angeles'],
    ['Charlie', 35, 'Chicago']
]
df_list = pd.DataFrame(data_list, columns=['Name', 'Age', 'City'])

# 3. From an External CSV File
# df_csv = pd.read_csv('data.csv')
```

Key Attributes of a DataFrame

Attribute	Description	Example Output
`df.shape`	Returns dimensions (rows, columns)	`(3, 3)`
`df.columns`	Returns column labels as an Index object	`Index(['Name', 'Age', 'City'])`
`df.index`	Returns row labels (Index object)	`RangeIndex(start=0, stop=3, step=1)`
`df.dtypes`	Returns data types of each column	`Name: object, Age: int64`
`df.size`	Returns total number of elements	`9`
`df.values`	Returns 2D NumPy array representation	`ndarray`

Accessing & Selection in DataFrames




```
# Select a Single Column (Returns a Series)
ages = df_dict['Age']

# Select Multiple Columns (Returns a DataFrame)
subset = df_dict[['Name', 'City']]

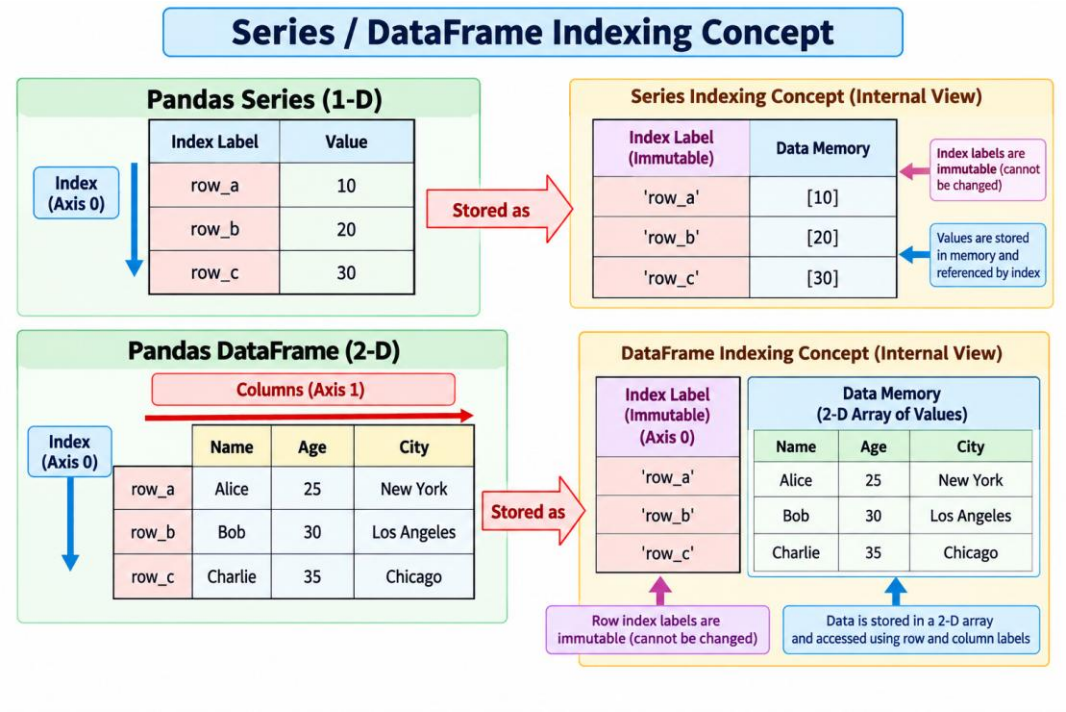
# Select Rows by Position (iloc)
first_row = df_dict.iloc[0] # First row

# Select Rows and Columns by Label (loc)
value = df_dict.loc[1, 'Name'] # 'Bob'

# Filtering Rows based on Condition
adults = df_dict[df_dict['Age'] >= 30]
```

3. Pandas Index Objects

The **Index** object in Pandas is responsible for holding axis labels and metadata (axis names, level names) for Series and DataFrames.



Key Properties of Index Objects

1. **Immutability**: Index objects are immutable and cannot be modified directly in place (e.g., `df.index[0] = 'new'` raises a `TypeError`). This ensures safe sharing across structures.
2. **Set Operations**: Behaves like a mathematical set, supporting operations such as `.union()`, `.intersection()`, and `.difference()`.
3. **Data Alignment**: Ensures that arithmetic operations align across matching labels, even if row orders differ.

Specialized Types of Indexes

- **`RangeIndex`**: Default memory-efficient integer index (`0, 1, 2, ...`).
- **`DatetimeIndex`**: Specialized index for time-series operations (e.g., timestamps, date ranges).
- **`MultiIndex`**: Hierarchical indexing structure allowing multi-level row/column axes.

4. Essential Functionality & Operations

Inspecting Data Structures

When working with new datasets, inspecting the dimensions and structure is the first step in exploratory analysis.

```
print(df.head(5))     # Returns first 5 rows
print(df.tail(5))     # Returns last 5 rows
print(df.info())      # Displays concise summary (dtypes, non-null counts)
print(df.describe())  # Calculates summary statistics for numerical columns
```

Handling Missing Data

Pandas data structures explicitly represent missing values using `NaN` (Not a Number) or `None`.

```
# Checking for missing values
print(df.isnull().sum())   # Count missing entries per column

# Filling missing values
df_filled = df.fillna(value={"Age": 0})

# Dropping missing rows
df_clean = df.dropna()
```

Adding, Modifying, and Removing Data

```
# Add a new column
```

```
df['Salary'] = [50000, 60000, 70000]

# Rename columns
df.rename(columns={'Age': 'Years'}, inplace=True)

# Drop a column
df.drop(columns=['City'], inplace=True)

# Drop a row by index
df.drop(index=0, inplace=True)
```

5. Comparative Summary Matrix

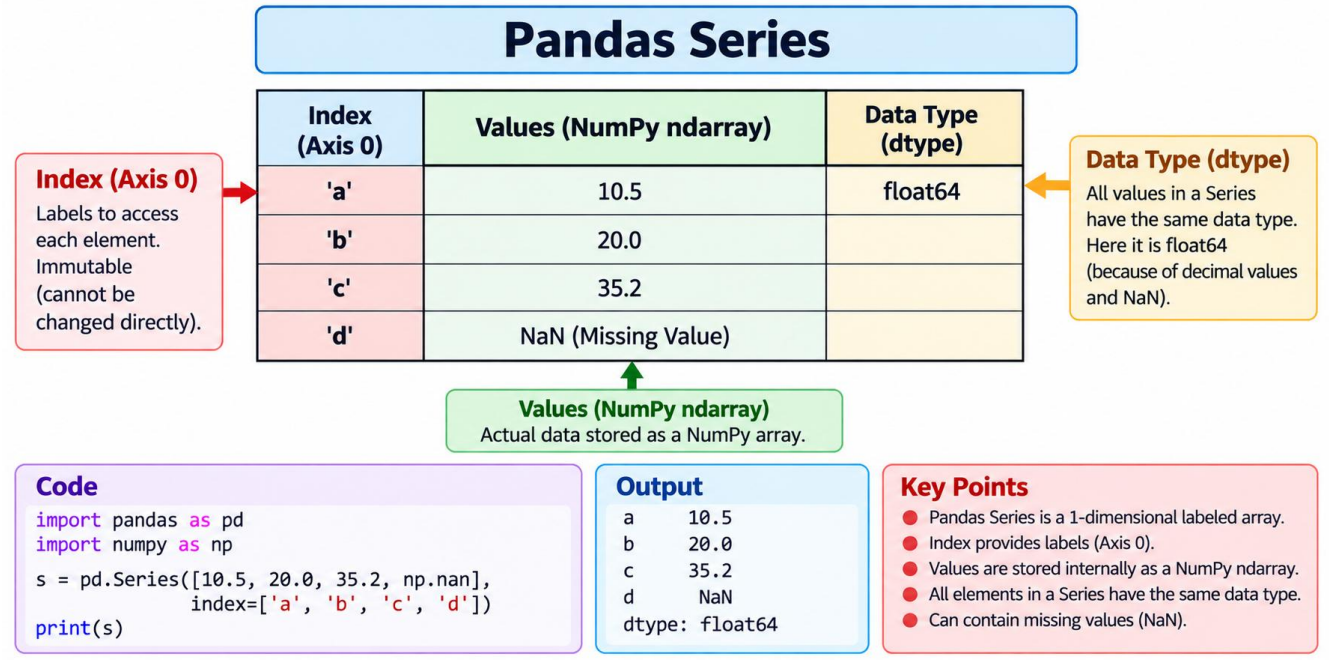
Feature	Pandas Series	Pandas DataFrame
Dimensions	1D (One-Dimensional)	2D (Two-Dimensional)
Data Types	Homogeneous (single type)	Heterogeneous (columns vary)
Analogy	Single Column / Vector / Dict	Spreadsheet / SQL Table / Matrix
Axes	1 Axis (`index`)	2 Axes (`index` for rows, `columns` for cols)
Creation Inputs	List, Dict, Scalar, NumPy array	Dict of Lists, List of Dicts, 2D NumPy Array, CSV
Size Mutability	Immutable size (requires re-creation)	Mutable (rows/columns added or removed easily)

Deep Dive: Pandas Series

A **Pandas Series** is a one-dimensional array-like object containing a sequence of values of a single data type and an associated array of data labels, called its **index**. It is one of the foundational data structures in Pandas and serves as the building block for DataFrames.

1. Conceptual Structure & Layout

Unlike a standard Python list or a 1D NumPy array, a Series pairs each data element with an explicit index label. This enables label-based lookup, automatic data alignment, and fast vectorised operations.



2. Methods of Creating a Series

Pandas offers several flexible options for instantiating a Series depending on the input data source:

- **From a Python List or Tuple:** Generates a default integer index (`0, 1, ..., N-1`) unless custom index labels are explicitly provided.
- **From a Python Dictionary:** Keys automatically become the index labels, and dictionary values become the Series data.
- **From a NumPy 1D Array:** Shares or copies underlying memory without requiring data conversions.
- **From a Scalar Value:** Broadcasts a single constant value across all specified index labels.

```
import pandas as pd
import numpy as np

# 1. List with explicit index
s_list = pd.Series([100, 200, 300], index=['Store_A', 'Store_B',
'Store_C'], name="Sales")

# 2. Dictionary
s_dict = pd.Series({'AAPL': 175.5, 'GOOGL': 140.2, 'MSFT': 380.1})

# 3. Scalar expansion
s_scalar = pd.Series(5.0, index=['r1', 'r2', 'r3', 'r4'])
```

3. Key Attributes & Properties

The following properties allow quick inspection of a Series instance:

Attribute	Type / Return	Description
`s.values`	`ndarray / ExtensionArray`	Returns the raw array of values without index labels.
`s.index`	`Index`	Returns the explicit axis index object.

Attribute	Type / Return	Description
`s.dtype`	`dtype`	Returns the data type of the contained values.
`s.name`	`str / hashable`	Optional identifier name for the Series (becomes column header in DataFrame).
`s.is_unique`	`bool`	Indicates whether all values in the Series are unique.
`s.nbytes`	`int`	Returns memory bytes consumed by the values.

4. Indexing, Selection, and Slicing

Pandas provides dedicated accessors to avoid ambiguity between label-based and integer position-based indexing:

- **Label-Based Selection (`s.loc[...]`):** Selects elements using index labels. When slicing using labels (e.g. `s.loc['a':'c']`), both start and end boundaries are **inclusive**.
- **Positional Selection (`s.iloc[...]`):** Selects elements using 0-based integer positions. Standard Python slicing semantics apply (end boundary is **exclusive**).
- **Boolean Masking:** Filters entries based on conditional statements returning boolean Series.

```
s = pd.Series([10, 20, 30, 40], index=['w', 'x', 'y', 'z'])

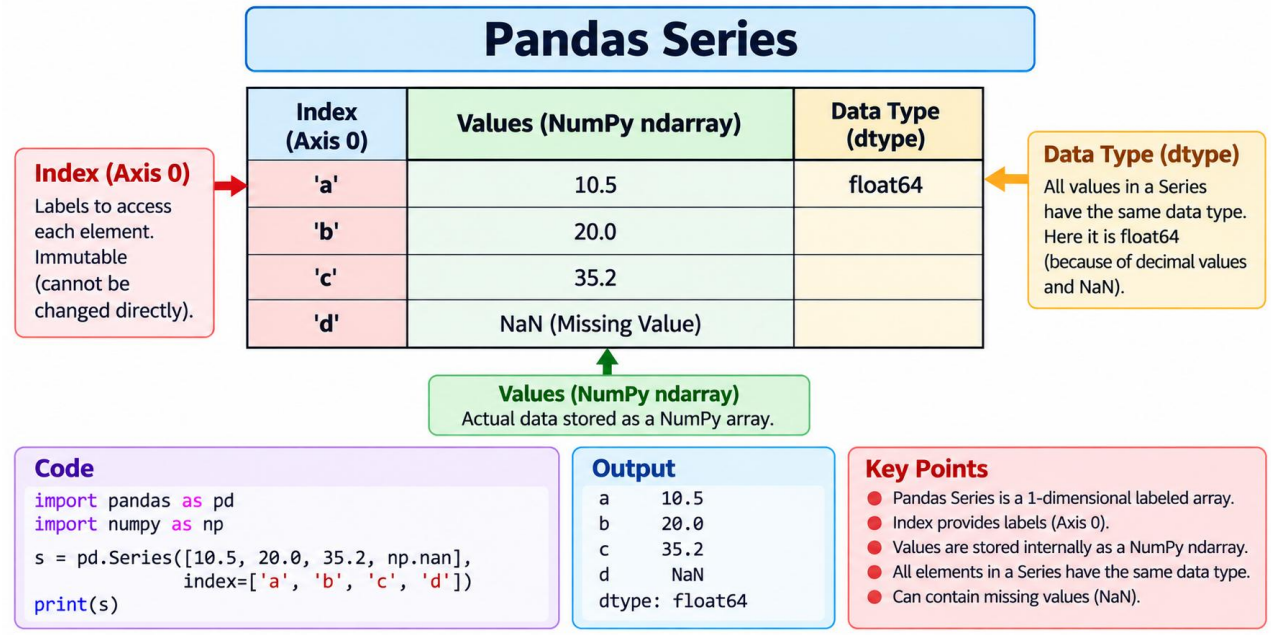
# Label indexing
val_x = s.loc['x']            # 20
slice_lbl = s.loc['x':'z']     # Includes 'x', 'y', 'z'
```

```
# Positional indexing
val_0 = s.iloc[0]              # 10
slice_pos = s.iloc[1:3]         # Includes positions 1 and 2 ('x', 'y')

# Boolean filter
filtered = s[s > 25]            # Returns 'y' (30) and 'z' (40)
```

5. Data Alignment & Operations

A fundamental feature of Pandas is **automatic index alignment**. When performing binary operations (addition, subtraction, multiplication, etc.) between two Series, Pandas automatically aligns matching index labels. Non-overlapping labels yield `NaN` (missing values).




```
s1 = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
s2 = pd.Series([10, 30, 40], index=['a', 'c', 'd'])

# Automatic alignment addition
result = s1 + s2

# Addition with default fill value for missing labels
```

```
result_filled = s1.add(s2, fill_value=0)
```

Deep Dive: Pandas DataFrames

A **Pandas DataFrame** is a two-dimensional, size-mutable, tabular data structure with labeled axes (rows and columns). It is the most commonly used Pandas object and can be thought of as a spreadsheet or SQL table.

1. Anatomy & Structure of a DataFrame

A DataFrame consists of three principal components: the data values, row index labels (Axis 0), and column name labels (Axis 1). Each individual column in a DataFrame is represented as a 1D Pandas Series sharing a common index.

2. Methods of Creating a DataFrame

DataFrames can be constructed from various data sources including dictionaries, list of lists, 2D NumPy arrays, or external files like CSV and Excel:

- **From a Dictionary of Lists:** Dictionary keys automatically map to column headers, while lists represent the column values.
- **From a List of Lists:** Requires explicitly passing the `columns` parameter to specify headers.
- **From External Files:** Read directly using functions like `pd.read_csv()` or `pd.read_excel()`.

```
import pandas as pd

# 1. From a Dictionary
data = {'Name': ['Alice', 'Bob', 'Charlie'], 'Age': [25, 30, 35],
'City': ['New York', 'Los Angeles', 'Chicago']}
df = pd.DataFrame(data)

# 2. From a List of Lists
list_data = [['Alice', 25], ['Bob', 30]]
df_list = pd.DataFrame(list_data, columns=['Name', 'Age'])
```

3. Key Attributes & Properties

Attribute	Type / Return	Description
`df.shape`	`tuple`	Returns tuple representing dimensionality (rows, columns).
`df.columns`	`Index`	Returns the column labels of the DataFrame.
`df.dtypes`	`Series`	Returns the data types of each column.
`df.index`	`Index`	Returns the row index labels.

4. Indexing, Selection, and Modification

Pandas provides clear accessors for selecting and modifying subset data:

```
# Column selection
ages = df['Age']
subset = df[['Name', 'City']]

# Row selection
row_by_pos = df.iloc[1]   # Second row by positional index
adults = df[df['Age'] > 28] # Conditional filtering

# Adding and updating columns
df['Salary'] = [50000, 60000, 70000]
df.loc[1, 'Age'] = 32 # Updating specific values
```

Deep Dive: Selection & Filtering Methods in Pandas

Data selection and filtering are among the most frequent operations when working with Pandas Series and DataFrames. Pandas provides multiple indexing mechanisms, each tailored for specific performance, syntactic convenience, or functional requirements.

1. Overview of Selection Operators & Indexers

Pandas primary indexing accessors and operators can be categorized as follows:

- **Indexing Operator (`[]`):** Primary interface for column selection, boolean filtering, and basic row slicing.
- **Label-Based Indexer (`loc[]`):** Accesses rows and columns using explicit row/column labels or boolean arrays.
- **Positional Indexer (`iloc[]`):** Accesses rows and columns using 0-based integer position index.
- **Fast Scalar Accessors (`at[]` and `iat[]`):** Optimized low-overhead lookup tools for extracting individual scalar values.
- **Expression Filtering (`query()`):** String-expression based filtering engine for clean and readable conditional queries.

2. Comparison Matrix of Accessors

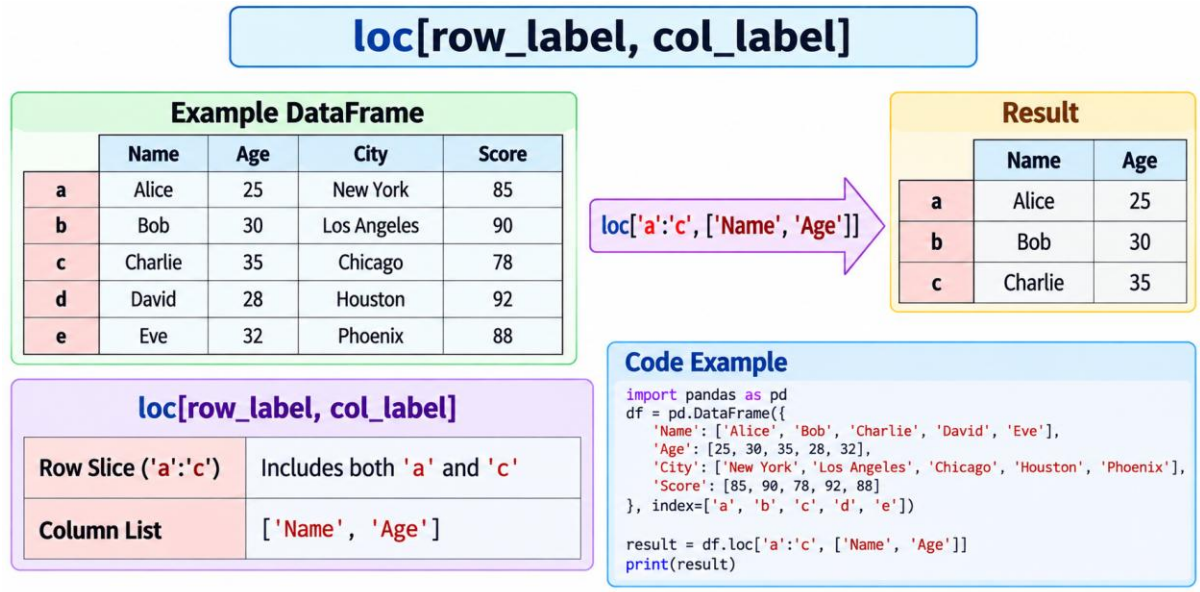
Method	Target Type	Slicing End Boundary	Use Case / Performance
`loc[]`	Labels / Names	**Inclusive** (Includes stop label)	General purpose selection by explicit label.
`iloc[]`	Integer Positions	**Exclusive** (Standard Python)	Position-based lookups and numerical slicing.
`at[]`	Single Label	N/A (Scalar only)	Fastest single value label lookup.
`iat[]`	Single Position	N/A (Scalar only)	Fastest single value positional lookup.

Method	Target Type	Slicing End Boundary	Use Case / Performance
`query()`	String Expression	N/A	Complex multi-condition filtering with concise syntax.

3. Detailed Breakdown & Code Examples

A. Label-Based Selection: loc[] vs. at[]

The `loc[]` accessor selects rows and columns using labels or boolean vectors. For scalar values, `at[]` provides lower overhead execution.




```
import pandas as pd
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, 30, 35, 40],
    'Score': [85, 92, 78, 88]
}, index=['r1', 'r2', 'r3', 'r4'])

# Label slicing (includes stop boundary 'r3')
subset_loc = df.loc['r1':'r3', ['Name', 'Score']]
```

```
# Single scalar value lookup using at[]
val_at = df.at['r2', 'Score']  # Returns 92
```

B. Positional Selection: iloc[] vs. iat[]

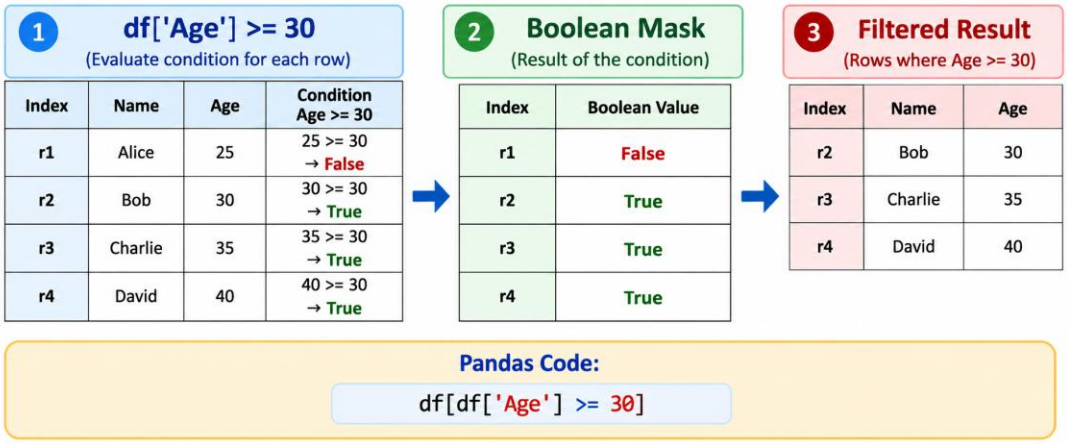
The `iloc[]` indexer allows integer-based location indexing from 0 to length-1 of the axis. Slicing mimics Python list semantics where the end position is excluded.

```
# Integer positional slicing (excludes row index 2)
subset_iloc = df.iloc[0:2, 0:2]  # Rows 0,1 and Cols 0,1

# Fast scalar positional lookup
val_iat = df.iat[1, 2]  # Returns row position 1, col position 2 (92)
```

C. Boolean Indexing

Boolean filtering evaluates logical conditions against DataFrame columns to produce a boolean Series mask, which filters rows where the condition evaluates to `True`.




```
# Multiple conditions using bitwise operators (&, |, ~)
mask = (df['Age'] >= 30) & (df['Score'] > 80)
filtered_df = df[mask]

# Using isin() for categorical filtering
filtered_isin = df[df['Name'].isin(['Alice', 'David'])]
```

D. Query Method: query()

The `query()` method uses string expression parsing for cleaner syntax, especially with multiple logical criteria. External Python variables can be referenced using the `@` symbol.

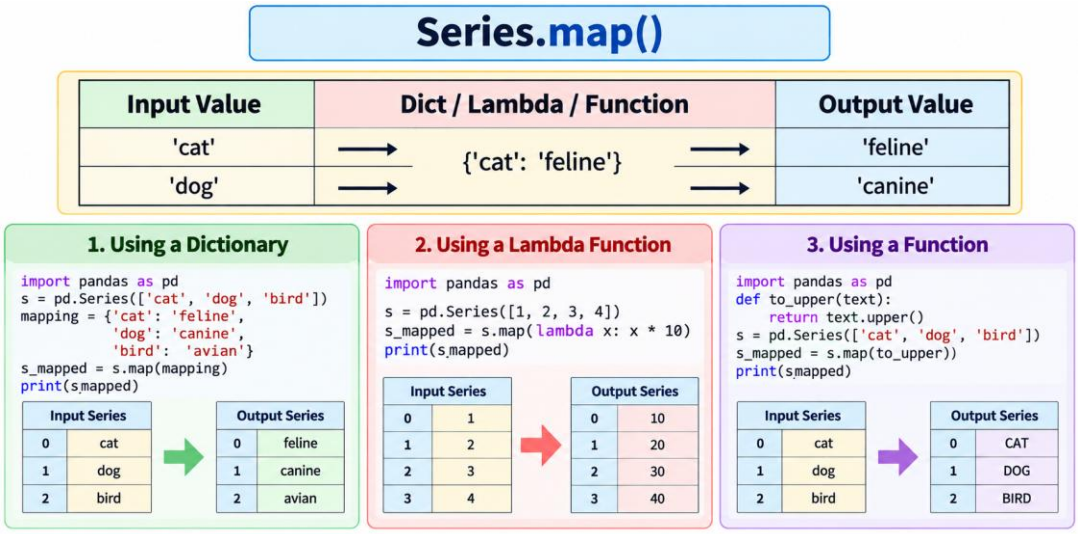
```
min_age = 30
# Clean syntax filtering with variable reference
query_res = df.query("Age >= @min_age and Score > 80")
```

Function Application and Mapping

Pandas provides powerful methods for applying custom or built-in functions across Series and DataFrames. Understanding when to use `map()`, `apply()`, `map() / applymap()`, and `transform()` is essential for efficient data transformation workflows.

1. Series.map(): Element-wise Mapping for Series

The `Series.map()` method is used to map values of a Series according to input correspondence, such as a dictionary, function, or another Series. It is strictly element-wise and operates only on 1D Series objects.



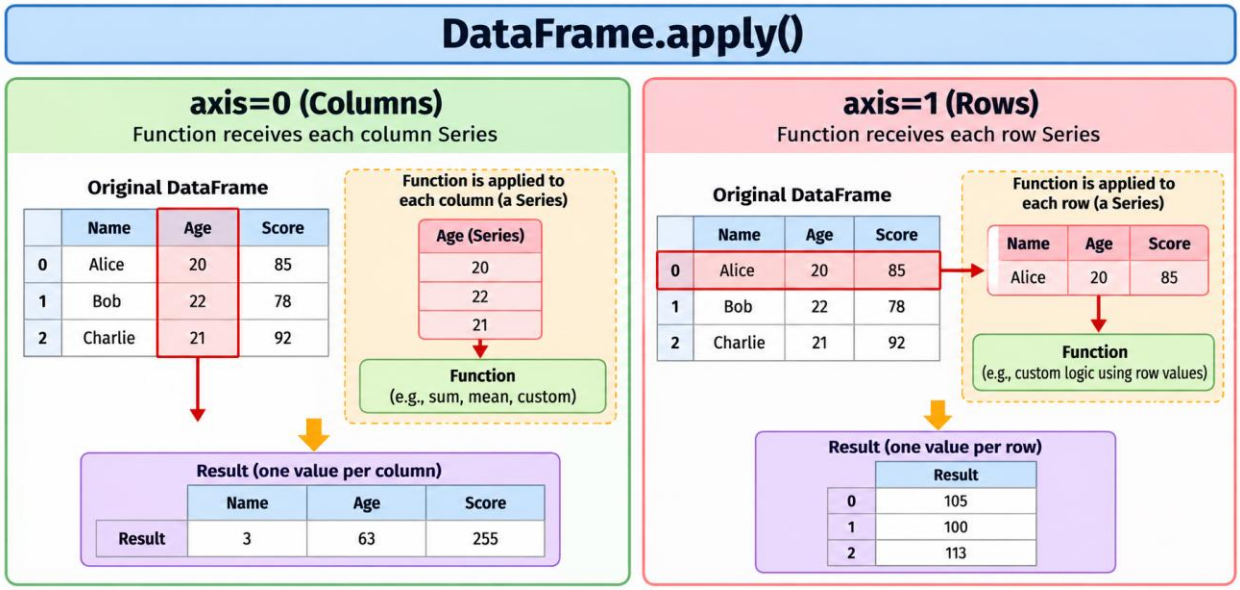

```
import pandas as pd
s = pd.Series(['cat', 'dog', 'cow', 'dog'])

# 1. Mapping using a dictionary
mapped_dict = s.map({'cat': 'feline', 'dog': 'canine'})
```

```
# 2. Mapping using a lambda function
mapped_func = s.map(lambda x: f"animal_{x}")
```

2. DataFrame.apply(): Axis-wise Function Application

The `DataFrame.apply()` method applies a function along an axis of the DataFrame (either rows or columns). It passes each column or row as a Series object to the applied function, allowing aggregate and row/column-level calculations.




```
df = pd.DataFrame({'A': [1, 2, 3], 'B': [10, 20, 30]})

# Column-wise sum (axis=0)
col_sums = df.apply(lambda col: col.sum(), axis=0)

# Row-wise custom computation (axis=1)
row_diff = df.apply(lambda row: row['B'] - row['A'], axis=1)
```

3. DataFrame.map() / applymap(): Element-wise Mapping for DataFrames

To apply a function element-wise across an entire DataFrame, use `DataFrame.map()` (introduced in Pandas 2.1.0 as a replacement for the deprecated `applymap()`). Every cell is transformed individually.

```
df = pd.DataFrame({'A': [1.234, 5.678], 'B': [9.101, 2.345]})

# Format every element to 1 decimal place
formatted_df = df.map(lambda x: f"{x:.1f}")
```

4. transform(): Shape-Preserving Transformations

The `transform()` method applies functions that must return an object of the same shape as the input. It is particularly useful when combined with `groupby()` operations to perform broadcasted group-level calculations without altering original DataFrame dimensions.

```
df = pd.DataFrame({
    'Group': ['A', 'A', 'B', 'B'],
    'Value': [10, 20, 100, 200]
})

# Normalize values by subtracting group means (returns same shape as original)
df['Group_Mean'] = df.groupby('Group')['Value'].transform('mean')
df['Normalized'] = df['Value'] - df['Group_Mean']
```

Sorting and Ranking in Pandas

Sorting and ranking are essential operations for data exploration, cleaning, and reporting. Pandas provides efficient built-in functions to sort data structures by index labels or column values, as well as computing numerical ranks for values.

1. Sorting by Index: sort_index()

The `sort_index()` method sorts a Pandas Series or DataFrame based on its index labels along a specified axis.

- **ascending:** Boolean flag (default `True`) to control sort order.
- **axis:** Sort along rows (`axis=0`) or columns (`axis=1`).

```
import pandas as pd
df = pd.DataFrame({'B': [2, 1, 3], 'A': [4, 5, 6]}, index=['z', 'a', 'k'])

# Sort by row index
sorted_rows = df.sort_index(ascending=True)

# Sort by column labels
sorted_cols = df.sort_index(axis=1)
```

2. Sorting by Values: sort_values()

The `sort_values()` method reorders data based on values within one or more columns or Series entries.

Series and Multi-Column DataFrame Sorting

When sorting DataFrames, pass a list of column names to the `by` parameter. You can also specify different sort directions for each column by supplying a list of booleans to `ascending`.

Handling Missing Values (NaN)

The `na_position` parameter determines where missing values are placed during sorting: `'last'` (default) puts `NaN` values at the end, while `'first'` places them at the beginning.

```
df = pd.DataFrame({
    'Dept': ['HR', 'IT', 'IT', 'HR'],
    'Salary': [50000, 75000, None, 60000]
})

# Multi-column sort with missing value positioning
```

```
sorted_df = df.sort_values(
    by=['Dept', 'Salary'],
    ascending=[True, False],
    na_position='last'
)
```

3. Computing Ranks: rank()

The `rank()` method assigns numerical ranks (1 through N) to data entries. When equal values exist, the `method` parameter determines how ties are resolved.

Tie Method	Description	Example Output for [10, 20, 20, 30]
`average`	Assigns average rank of tied values (default)	`[1.0, 2.5, 2.5, 4.0]`
`min`	Assigns minimum rank among tied group	`[1.0, 2.0, 2.0, 4.0]`
`max`	Assigns maximum rank among tied group	`[1.0, 3.0, 3.0, 4.0]`
`first`	Assigns ranks in order values appear in data	`[1.0, 2.0, 3.0, 4.0]`
`dense`	Like 'min', but rank increases by 1 for next group (no gaps)	`[1.0, 2.0, 2.0, 3.0]`

```
s = pd.Series([10, 20, 20, 30])

# Rank computation examples
rank_avg = s.rank(method='average')
rank_dense = s.rank(method='dense')
```

4. Summary Matrix of Sorting & Ranking Methods

Method	Primary Purpose	Key Parameters
`sort_index()`	Order by row/column index labels	`axis, ascending, inplace`
`sort_values()`	Order by values in columns or Series	`by, ascending, na_position, inplace`
`rank()`	Calculate numerical ranks for data entries	`method, ascending, na_option, pct`

Summarizing Data in Pandas

Summarizing data is a critical step in exploratory data analysis (EDA). Pandas provides an extensive suite of built-in functions to compute descriptive statistics, locate minimum and maximum values, generate comprehensive statistical summaries, and analyze categorical frequency distributions across Series and DataFrames.

1. Core Statistical Aggregations

Pandas offers high-performance computational methods for standard summary metrics. These functions can be applied directly to a Series or along specific axes of a DataFrame (defaulting to columns, `axis=0`).

- **`sum()`**: Computes the sum of values.
- **`mean()`**: Calculates the arithmetic mean (average).
- **`median()`**: Finds the 50th percentile (middle value).
- **`min()` / `max()`**: Returns the minimum and maximum values.
- **`std()`**: Computes the sample standard deviation (normalized by N-1).

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'Sales': [100, 250, 150, 400, np.nan],
```

```
    'Profit': [20, 50, 30, 80, 10]
})

# Core aggregation functions (missing values are excluded by default)
print("Total Sales:", df['Sales'].sum())
print("Mean Sales:", df['Sales'].mean())
print("Median Sales:", df['Sales'].median())
print("Min/Max Profit:", df['Profit'].min(), df['Profit'].max())
print("Std Dev Profit:", df['Profit'].std())
```

2. Extrema Location: idxmin() and idxmax()

While `min()` and `max()` return the extreme values themselves, `idxmin()` and `idxmax()` return the index labels corresponding to where those minimum or maximum values occur.

```
df_sales = pd.DataFrame({
    'Revenue': [12000, 45000, 28000, 51000]
}, index=['Q1', 'Q2', 'Q3', 'Q4'])

# Find index labels of minimum and maximum revenue
lowest_qtr = df_sales['Revenue'].idxmin()   # Returns 'Q1'
highest_qtr = df_sales['Revenue'].idxmax()  # Returns 'Q4'
```

3. Statistical Summaries: describe()

The `describe()` method generates summary statistics that summarize the central tendency, dispersion, and shape of a dataset's distribution, excluding missing values.

```
# Descriptive statistics for numerical columns
print(df.describe())

# Summary for categorical/object data
s_cat = pd.Series(['A', 'B', 'A', 'C', 'B', 'A'])
print(s_cat.describe())
```

4. Frequency Distribution Analysis

Analyzing categorical distributions involves inspecting unique elements and counting value occurrences across a Series.

- **`unique()`**: Returns an array of unique values in order of appearance.
- **`nunique()`**: Returns the count of distinct elements (excluding NaNs by default).
- **`value_counts()`**: Computes frequency counts for unique values, sorted in descending order.

```
cities = pd.Series(['NY', 'LA', 'NY', 'Chicago', 'LA', 'NY', None])

# Unique values array and total count
print("Unique cities:", cities.unique())
print("Count of unique cities:", cities.nunique())

# Frequency distribution count
print(cities.value_counts())

# Normalized relative frequencies (percentages)
print(cities.value_counts(normalize=True))
```

Unique Values and Value Counts in Pandas

Analyzing categorical and discrete data often requires identifying distinct elements and calculating their frequency distributions. Pandas provides three primary methods for these tasks: `unique()`, `nunique()`, and `value_counts()`.

1. Extracting Unique Elements: unique()

The `Series.unique()` method extracts all unique values from a 1D Series object in order of appearance. It returns a NumPy array or ExtensionArray of unique values, including missing values (`NaN`).

```
import pandas as pd
import numpy as np
```

```
cities = pd.Series(['New York', 'Los Angeles', 'New York', 'Chicago',
'Los Angeles', np.nan])

# Get unique elements in order of appearance
unique_cities = cities.unique()
print(unique_cities)
```

2. Counting Unique Elements: nunique()

The `nunique()` method counts the number of distinct non-null entries. It can be applied to both Series and DataFrame objects.

- **`dropna`**: Boolean flag (default `True`) to exclude `NaN` from the count.
- **`axis`**: When used on DataFrames, determines whether to count along columns (`axis=0`) or rows (`axis=1`).

```
# Count distinct non-null elements in a Series
count_no_nan = cities.nunique(dropna=True)
print("Distinct cities (excluding NaN):", count_no_nan)

# Count including NaN values
count_with_nan = cities.nunique(dropna=False)
print("Distinct cities (including NaN):", count_with_nan)

# DataFrame level unique counts
df = pd.DataFrame({
    'Department': ['HR', 'IT', 'IT', 'Sales'],
    'Role': ['Manager', 'Developer', 'Developer', 'Lead']
})
print(df.nunique())
```

3. Computing Frequency Distributions: value_counts()

The `value_counts()` method returns a Series containing counts of unique values in descending order. Key parameters include:

- **`normalize`**: Returns relative frequencies/percentages rather than absolute counts when set to `True`.
- **`sort & ascending`**: Controls whether results are sorted by frequency (default `sort=True`) and the order (`ascending=False`).
- **`dropna`**: Includes counts for missing values if set to `False`.
- **`bins`**: Groups continuous numerical data into discrete intervals.

```
scores = pd.Series([85, 90, 85, 78, 90, 85, np.nan])

# Absolute value counts including NaNs
print(scores.value_counts(dropna=False))

# Relative proportions (normalized)
print(scores.value_counts(normalize=True))

# Binning continuous values into 3 equal-width intervals
print(scores.value_counts(bins=3))
```

Membership Operations in Pandas

Membership operations allow you to check whether elements exist within a specific container or dataset, filter DataFrames or Series based on set membership, and isolate matching records efficiently. In Pandas, membership testing is primarily performed using the `isin()` method, index membership checks, or string-based containment methods.

1. Series Membership with isin()

The `isin()` method evaluates each element in a Series and returns a boolean Series indicating whether each value matches any value in a provided list, set, or array.

```
import pandas as pd

# Create a Series of fruits
fruits = pd.Series(['apple', 'banana', 'cherry', 'date',
'elderberry'])
```

```
# Define a set of target items
target_fruits = ['apple', 'cherry', 'fig']

# Check membership
mask = fruits.isin(target_fruits)
print("Boolean Mask:")
print(mask)

# Filter Series using membership mask
matching_fruits = fruits[mask]
print("\nFiltered Fruits:")
print(matching_fruits)
```

2. DataFrame Membership Filtering

When applied to a DataFrame column, `isin()` provides a clean, vectorised way to filter rows matching multiple target categories without needing repeated equality (`==`) or logical OR (`|`) conditions.

```
import pandas as pd

# Create sample DataFrame
df = pd.DataFrame({
    'Employee': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'Department': ['HR', 'IT', 'Finance', 'IT', 'HR'],
    'Salary': [60000, 85000, 95000, 78000, 62000]
})

# Filter for employees in HR or Finance
target_depts = ['HR', 'Finance']
filtered_df = df[df['Department'].isin(target_depts)]
print("Employees in HR or Finance:")
print(filtered_df)
```

3. Multi-Column Membership with Dictionary Mapping

Passing a dictionary to `df.isin()` allows simultaneous membership checking across multiple columns, where dictionary keys represent column headers and values represent list-like targets.

```
import pandas as pd

df = pd.DataFrame({
    'Department': ['HR', 'IT', 'Finance', 'IT', 'HR'],
    'Salary': [60000, 85000, 95000, 78000, 60000]
})

# Define multi-column membership criteria
criteria = {
    'Department': ['HR', 'IT'],
    'Salary': [60000, 85000]
}

# Returns boolean DataFrame
bool_df = df.isin(criteria)
print("Multi-column Membership Mask:")
print(bool_df)

# Filter rows where all conditions match using all(axis=1)
match_all = df[df.isin(criteria).all(axis=1)]
print("\nRows matching all criteria:")
print(match_all)
```

4. Inverse Membership (Negation operator ~)

To exclude specific membership matches, use the bitwise NOT operator (~) to negate the boolean mask returned by `isin()`.

```
import pandas as pd

df = pd.DataFrame({
```

```
    'Product': ['Laptop', 'Mouse', 'Monitor', 'Keyboard'],
    'Category': ['Tech', 'Accessory', 'Tech', 'Accessory']
})

# Exclude accessories
non_accessories = df[~df['Category'].isin(['Accessory'])]
print("Non-accessory Products:")
print(non_accessories)
```

5. Index and String Membership Operations

Membership testing can also be applied to Index objects using `Index.isin()` or Python's `in` keyword, as well as checking substring occurrences using `str.contains()`.

```
import pandas as pd

# A. Index membership check
idx = pd.Index(['r1', 'r2', 'r3', 'r4'])
print("Is 'r2' in index?", 'r2' in idx)
print("Index mask:", idx.isin(['r1', 'r4']))

# B. String substring containment
series_str = pd.Series(['Data Science', 'Machine Learning', 'Deep
Learning'])
has_learning = series_str.str.contains('Learning')
print("\nSubstring match mask:")
print(has_learning)
```

Reading and Writing Data in Text Format in Pandas

Interacting with external text files is a foundational task in data science workflows. Pandas provides robust tools for loading tabular datasets from text formats (such as CSV and delimited text files) into DataFrames, as well as exporting modified DataFrames back to disk.

1. Primary Reading Functions: read_csv() & read_table()

Pandas offers two primary functions for parsing text-based tabular datasets:

- **`pd.read_csv()`**: Specifically tailored for comma-separated value files (default delimiter is `','`).
- **`pd.read_table()`**: General-purpose reader for tab-separated or custom-delimited files (default delimiter is `'\t'`).

```
import pandas as pd

# Reading a standard comma-separated CSV file
df_csv = pd.read_csv('dataset.csv')

# Reading a tab-separated text file using read_table()
df_tsv = pd.read_table('dataset.tsv')
```

2. Key Parsing & Loading Parameters

The reading functions accept a wide array of options to handle varied file structures, custom delimiters, missing values, and memory optimizations:

Parameter	Description	Example Usage
`sep / delimiter`	Delimiter character or regular expression string	`sep=';' or sep=r'\s+'`
`header`	Row number(s) to use as column names (use `None` if no header present)	`header=None or header=0`
`index_col`	Column name or position index to use as row labels	`index_col='ID' or index_col=0`
`usecols`	Subset of columns to load into memory	`usecols=['Name', 'Salary']`

Parameter	Description	Example Usage
`dtype`	Data type specifications for specific columns	`dtype={'Age': 'int32', 'ZIP': 'str'}`
`na_values`	Additional custom strings to recognize as missing/NaN values	`na_values=['NA', 'Missing', 'N/A', '-999']`
`skiprows`	Line numbers or total count of lines to skip at file beginning	`skiprows=5 or skiprows=[0, 2, 5]`

```
import pandas as pd

# Advanced reading configuration with multiple parsing parameters
df = pd.read_csv(
    'employee_data.csv',
    sep=',',
    header=0,
    index_col='Emp_ID',
    usecols=['Emp_ID', 'Name', 'Department', 'Salary'],
    dtype={'Salary': 'float64'},
    na_values=['N/A', 'Unknown', '-999'],
    skiprows=2
)
```

3. Writing Data to Text Format: to_csv()

To export DataFrames back into text files, use `to_csv()`. Important flags include `index=False` to prevent writeout of default integer row indices, `sep` for choosing target delimiters, and `na_rep` to customize how missing values are represented on disk.

```
import pandas as pd

df = pd.DataFrame({
```

```
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, None, 35],
    'City': ['New York', 'Los Angeles', 'Chicago']
})

# Export to CSV without row index labels
df.to_csv('output_clean.csv', index=False, na_rep='NULL')

# Export to tab-delimited text file
df.to_csv('output_tab.txt', sep='\t', index=False)
```